

Data Structure : organizes the data inside the memory (RAM/HDD) s.t.

- the space needed is compact;
- query/update is fast;
(# queries is huge)
- 1-time preprocessing can be slow, but not too slow.

Challenging Problem: Range-min in an array.

Query: $i \leq j$.

Output: $\min A[i, \dots j]$.

Range-Minima in 1-D Array

Ideas?: Brute-force either takes $O(n^2)$ -space & query = $O(1)$ -time
or it takes: $O(n)$ -space & query = $O(n)$ -time.

Qn: Design a ds to improve this tradeoff?

- Like max-sum-subarray solution, we can focus on just one index, say i .
 - ↳ What do we do with j 's now? They are still $O(n)$ many. $\therefore$ we need to account for every j in the query.
 - $\Rightarrow n \times O(n)$ space/time?

Idea: Consider going over j 's that are powers of 2 $\Rightarrow O(\lg n)$ many j 's !

- Tiny ds proposed: $B[i][k] = \min$ of $A[i, i+1, \dots, i+2^k]$, $\forall 0 \leq i \leq n-1$ & $0 \leq k \leq \lg n$.

- B is $n \times \lg n$ matrix (2D-array). $[|j-i| = 2^k]$

- Q_n : Is an arbitrary query (i, j') related to (i, j) 's where $|j-i| = 2$ -power?

- For arbitrary query (i, j') we can reduce to a different query (i, j) & read-off the answers from B.

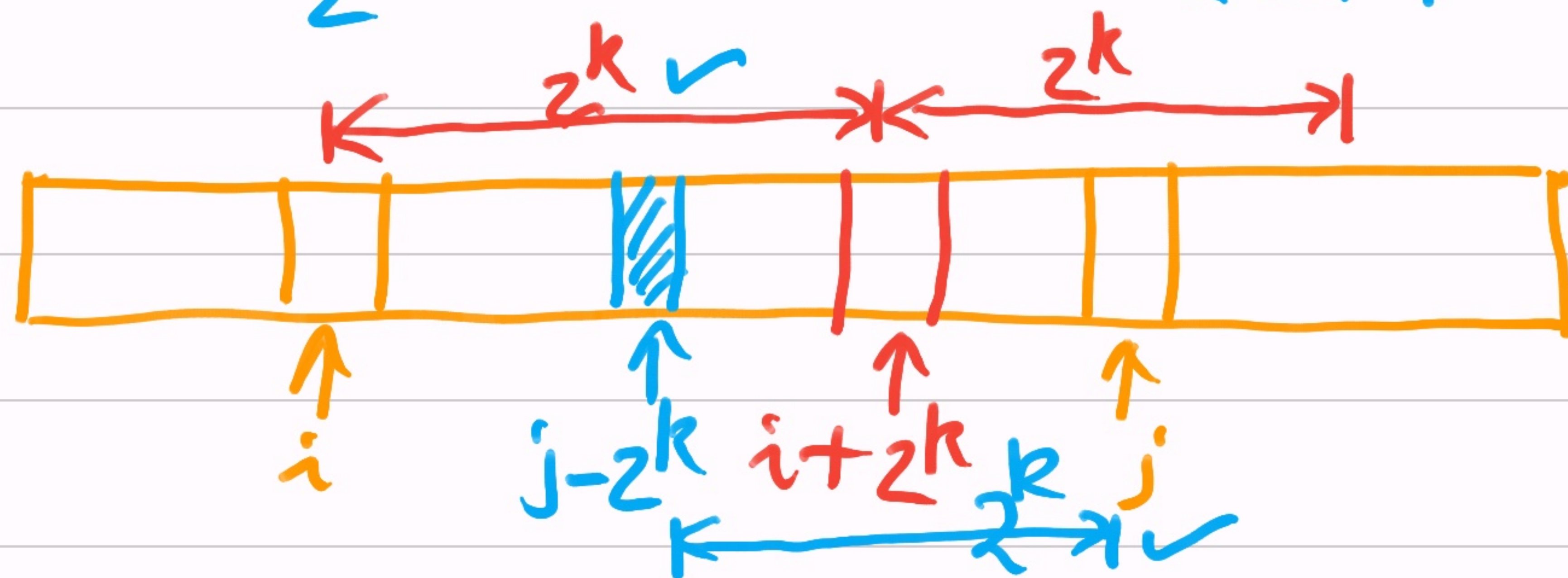
↳ Collaboration

- Define maxPow2(L) function as:
 $= 2^k$ s.t. $2^k \leq L < 2^{k+1}$.

▷ $\frac{L}{2} < \text{maxPow2}(L) \leq L$.

▷ Intervals
 $[i, i+2^k]$ &
 $[j-2^k, j]$ collaborate
to give $[i, j]$.

A:



- Pseudocode:

- Build $B[n][\lg n]$; $\text{// } |j-i| = 2\text{-power.}$
- $\text{rngMin}(A, i, j) \{ \text{// } i \leq j$
 - $L \leftarrow j - i;$
 - $t \leftarrow \text{maxPow2}(L);$ $\text{// } t \text{ can be stored in an array.}$
 - $k \leftarrow \lg(L);$ $\text{// } t = 2^k \leq L = j - i < 2^{k+1}$
 - $\{$
 - $\text{if}(t = L) \text{return } B[i][k];$
 - $\text{else return min}(B[i][k], B[j-t][k]);$
 - $\}$

$\triangleright$ #steps (1 query) $\leq O(1)$.
 storage space $\approx |B| \leq O(n \lg n)$.

▷ Building B takes #steps $\leq O(n^2 \lg n)$.

Pf: • $B[i][k] = \min A[i, i+1, \dots, i+2^k]$

can be done in $O(n)$ steps.

• total $\leq n \times \lg n \times O(n) = O(n^2 \lg n)$. ◻

Qn: Can B be built faster?

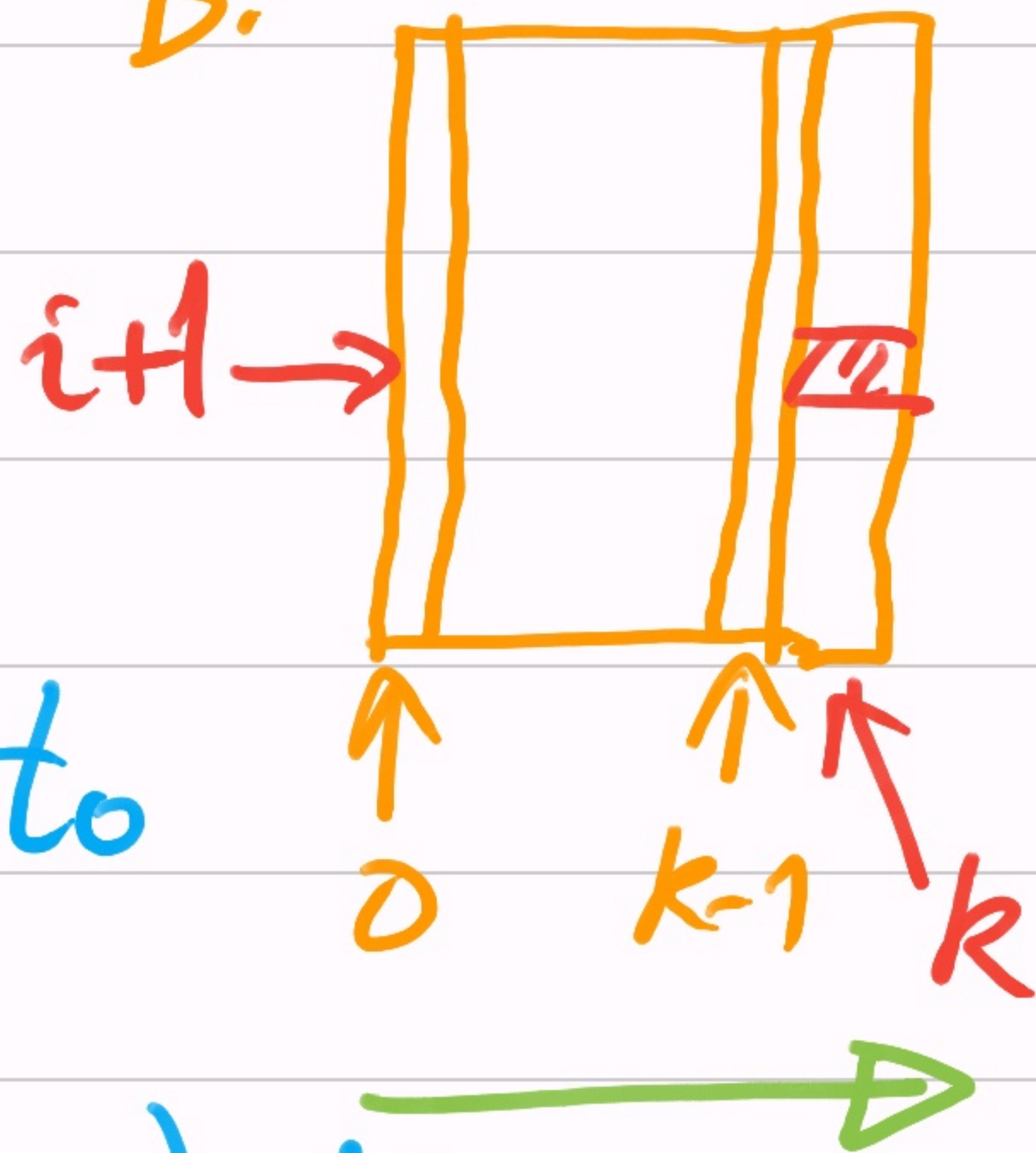
$$\begin{aligned} - B[i+1][k] &= \\ \min A[i+1, \dots, i+1+2^k] \\ &= \min (A[i+1, \dots, i+2^k], \\ &\quad A[i+1+2^k]) \end{aligned}$$



$$= \min(\underbrace{\text{rngMin}(A, i+1, i+2^k)}, A[i+2^k+1]).$$

the pseudocode need B up to the $(k-1)$ th col. only.

▷ This step needs time $\leq O(1)$ by 1-query of rngMin .



⇒ ▷ The k -th col. of B, using the earlier ones, takes $O(n)$ time to fill.

⇒ ▷ B can be built in $O(n \lg n)$ time.

Theorem: Range-Min in a 1-D array can be solved in: $O(n \log n)$ preprocessing, $O(1)$ query-time, and $O(n \log n)$ space.

Exercise: Write the final pseudocode.

- ↳ • Fill $B[*][0]$ (simply, min of two elts.)
- Iteratively, fill $B[*][k]$, using $B[*][0 \dots k-1]$; need to use the recurrence & rngMin queries with this partially-filled B matrix.

- DS design involves two broad steps:

Step 1: Mathematical Modeling.

Step 2: Implementation of the model.

- Step 1 is about giving the abstract description of the data format and the list of operations (query + update).

↳ The output is called ADT = Abstract Data Type.

- Step 2 is about implementing the ADT in a

Computer / program.

↳ The output is a program for preprocessing + query + update ops.

List - Modeling the List ADT.

- ex. • Roll nos. of students passing ESO207.
- Rooms reserved in a hotel.
- Students getting convocation awards.

What's common? → A list $L: a_1, a_2, \dots, a_i, \dots, a_n$.
 i -th-element a_i

Query Ops. on L : • $\text{isEmpty}(L)$: $L \stackrel{?}{=} \emptyset$?

• $\text{search}(x, L)$: $x \in L$?

• $\text{successor}(p, L)$: return the element that appears after location p in L .

• $\text{predecessor}(p, L)$: — before —

eg. $\text{successor}(i, L) = a_{i+1}$ or EOF
 $\text{predecessor} \quad " \quad = a_{i-1}$.

• $\text{first}(L)$: return a_1 .

• $\text{enumerate}(L)$: Print all the elements.

Update ops. on L:

- $\text{createEmpty}(L)$: Create space; pointer & initialize $L \leftarrow \phi$;
- $\text{insert}(x, p, L)$: Insert x at loc./ptr. p in L .

eg. $L = \{3, 6, 9\}$ & p pts. to 6 then
 $\text{insert}(x, p, L)$ gives : $L_{\text{new}} \leftarrow \{3, x, 6, 9\}$.

- $\text{delete}(p, L)$: Delete loc./ptr. p in L .
eg. $\text{delete}(p, L)$ gives $L_{\text{new}} \leftarrow \{3, 9\}$.

- $\text{makeEmpty}(L) : \text{Force } L \leftarrow \emptyset;$

- But, what's the fastest implementation?

Ideas? (i) Array-based implement.

- Put L in an array $A[\]$; store a_i in $A[i-1]$; $i = 0 \dots n-1$ for $n = |L|$.

▷ Finding the i -th element a_i in L is the same as outputting $A[i-1]$. It takes $O(1)$ time.

If: Array has Random-Access: A is a pointer,

and $(A+i)$ is a pointer to the cells
 $A[0]$ and $A[i]$ respectively.
 $\Rightarrow A[i-1]$ takes $O(1)$ time. $\square$

▷ Array locations are: $A, A+1, \dots, A+n-1$.
(in the address notations)

eg. $\text{int } A[] \Rightarrow A$ is (int^*) type.

▷ The complexity of all the ops. is:

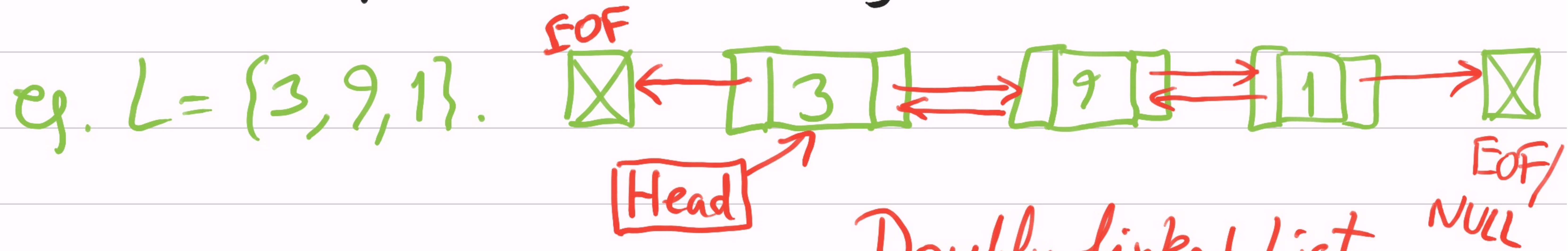
$\text{isEmpty}(L)$ $O(1)$

$\text{search}(x, L)$ $O(n)$ //unsorted A

Successor(i, L)	$O(1)$	
predecessor(i, L)	$O(1)$	// Check for EOF
createEmpty(L)	$O(1)$	
insert(x, i, L)	$O(n)$	// $A+i-1$ is known,
$A[i-1] \leftarrow x;$		// but, you shift $A[i-1, \dots, n-1]$
		to $A[i, \dots, n]$.
delete(i, L)	$O(n)$	
makeEmpty(L)	$O(1)$	.

— $\therefore$ Insertion/Deletion are important ops,
we need to optimize them,

(ii) Patters (C language) give a Link-based implementation of the List ADT:



▷ List L is accessible by the Head variable.
(eg. A var. in array)

- How to implement $\text{insert}(x, p, L)$?

Idea: Say, $p \rightarrow 9$: One needs to work with 3, p & x ; ignore the rest!

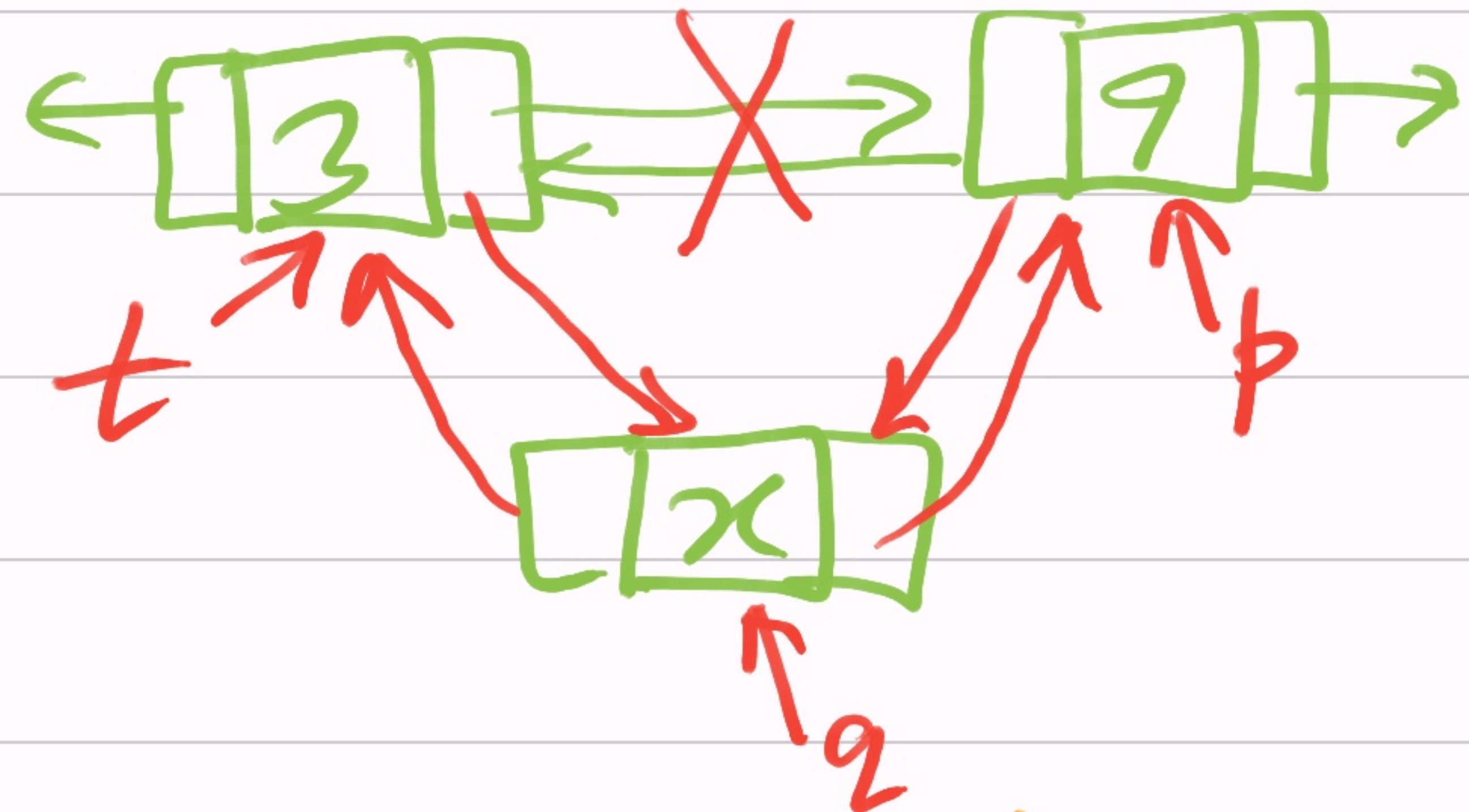
```

insert(x, p, L) {
  t ← p.left;
  q ← new(node);
  q.val ← x;
  q.left ← t; q.right ← p;
  t.right ← q; // completes t
  p.left ← q; // completes p
  // ⇒ L is updated to size = n+1
}

```

completes
q

node is a struct_



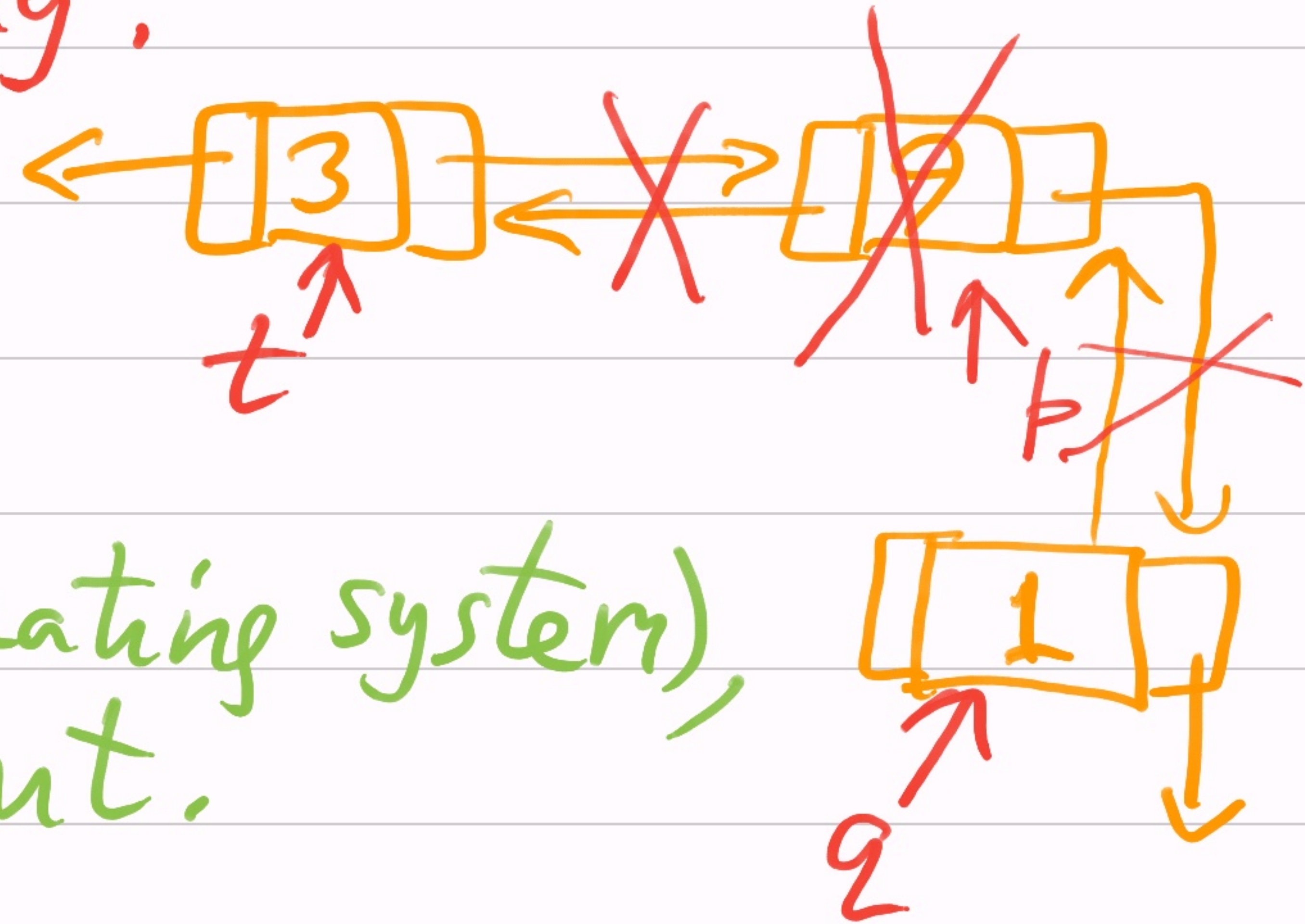
▷ insert(x, p, L) takes $O(1)$ time.

Qn: Why's this faster than the Array?

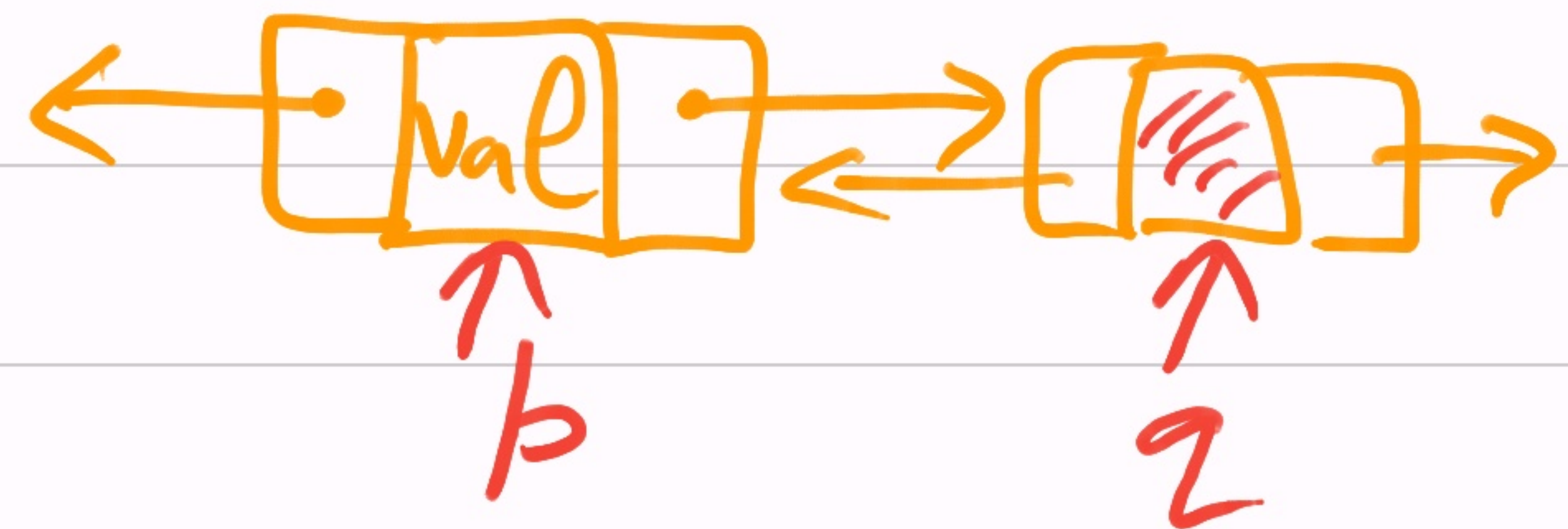
Exercise: delete(p, L) is implemented in a similar way.

▷ delete(p, L) is $O(1)$ time.

Note: The freed space is reused by the OS (operating system), which we won't care out.



- successor(p, L) ?
 $q \leftarrow p.\text{right};$
 return $q.\text{val};$ }



▷ Linked-list has time-complexity:

isEmpty(L)	$O(1)$
search(x, L)	$O(n)$
succ(p, L)	$O(1)$
pred(r)	$O(1)$
createEmpty(L)	$O(1)$
insert(x, p, L)	$O(1)$
delete(p, L)	$O(1)$
makeEmpty(L)	$O(1)$

// — — —
// Space reuse is
left to the OS.
↳ garbage-collection

Exercise: WAP to implement
these (e.g. in C).

- Example for List ADT :

Maintain a telephone directory as a list L with operations:

- Search name x
- Insert a new record (name, phone#)

array	links
$O(n)/O(\log n)$	$O(n)$
$O(n)/O(1)$	$O(1)$

Qn: Can we get the best of both implements?

- Maintain a sorted link-list to do binary-search. $\rightarrow$ Insert again requires $O(n)$ in list.